NY Waterway Fare Configuration Tool

Course: CMPT 276 — Introduction to Software Engineering, Simon Fraser University

Project Repository: https://github.com/harpoonwithaz/nyw-fare-config

Authors: Athan L, Harendeep, Lakshya J, Oliver T, Tejas D

Application Link: nyw-fare-config

Abstract

NY Waterway Fare Configuration Tool is an administrative web application designed to replace the fragile, manual configuration workflow currently used by Caracal Technologies and NY Waterway (NYW) to configure physical Ticket Vending Machines (TVMs) across ferry terminals. The application provides an interactive interface for client services staff and operators to update origin-destination fares, manage ticket availability and sales windows, author pre-purchase route alert banners, and publish vetted catalog configurations. The technical implementation will include a Java Spring Boot web application using Thymeleaf as the templating engine with a PostgreSQL database for storing session based credentials and persistent storage. The application communicates via REST HTTP calls with Caracal Technologies' Docker-packaged multi-tenant platform API and server.

Core Problem & How We Solve It

Background

Caracal Technologies manufactures and operates TVMs for parking facilities, parks and recreation, and municipal/mass transit clients, including New York Waterway (NYW). New York Waterway operates commuter ferry services across multiple terminals with distinct route-based pricing and operating schedules.

Current workflow & Existing Systems

NYW operational staff compile route updates, schedule changes, and rate adjustments into complex, two-dimensional Microsoft Excel spreadsheets. These spreadsheets are emailed to Caracal Technologies, where engineers manually translate spreadsheet rows into raw, deeply nested JSON configuration files. These spreadsheets provide no transactional integrity, no live synchronization with TVM fleet endpoints, no constraint checking, and require technical developers to execute updates.

Existing commercial fleet configuration suites (e.g., SAP, custom automated fare collection platforms) are costly, proprietary, and incompatible with Caracal's multi-tenant dimensional pricing architecture.

What Problem We Solve

The reliance on spreadsheets and manual process of updating the ferry ticket prices, availability, and customer warning messages is extremely time consuming. Our application aims to solve this problem by providing a centralized, user friendly interface where the employees can manage ticket configurations, review changes, and publish updates more efficiently and accurately. It will introduce localized persistence for pre-purchase passenger notices, and enforce strict front-end and server-side schema constraints (e.g., preventing negative dollar amounts or malformed schedules) at the point of entry.

Target Audience

This web application is an administrative utility tool engineered for enterprise mass-transit operations, replacing disconnected email threads and raw spreadsheet manipulation with a single operational source of truth. The primary target audience consists of Caracal Technologies client services personnel responsible for maintaining client configurations, with New York Waterway station operators and dispatchers serving as secondary stakeholders who require intuitive visibility into route availability and customer service alerts.

Project Scope & 5 Core Epics

Features and Sub-problems

The NY Waterway Fare Configuration Tool is structured not as disconnected features, but as a single, unified enterprise utility addressing five core sub-problems. To ensure equitable scope across all five team members, each engineer assumes end-to-end ownership over one architectural epic, spanning controller endpoints, data transfer validation, relational persistence, and user interface workflows.

Epic 1 : Operator Authentication & Role-Based Access Control

Lead: *Harendeep S*

The main focus of this epic is to provide NY Waterway employees with a secure and convenient way to access the system and to prevent unauthorized access.

In this epic, we will develop a login page using spring sessions and a PostgreSQL database, which would allow employees to sign in using their registered email address or username and password. In later iterations, we will develop a Google Login page which would allow employees to sign in using their authorized Google accounts to achieve a faster and easier login process.

In both of these iterations, we will carefully and strictly verify user credentials and ensure that only authorized employees can access the system using a role-based access control system. Verified operator emails will directly map to Caracal's `X-Actor: operator:<email>` API http header.

In case of unauthorized accounts, invalid credentials, missing fields, and authentication failures, we will also display appropriate error messages.

We will also implement a logout system which logs the user out for improved security and also auto-logout after a set time. The main goal is to provide NY Waterway employees with a secure, user-friendly authentication experience through both session credential login and Google login, ensuring that only authorized users can access the system.

Epic 2 : Interactive Origin-Destination Rate Grid & Fare Management

Lead: *Oliver T*

This epic addresses the core daily task of NY Waterway fare managers by delivering an interactive rate table that mirrors the operator's mental model. It replaces the error-prone manual translation of spreadsheets into raw JSON configuration files by providing an intuitive, web-based rate management matrix for Caracal client services personnel and New York Waterway administrators.

When an operator selects an origin terminal, the interface dynamically constructs a comprehensive interactive pricing grid displaying every valid route destination alongside all ticket products and their prevailing prices. Each row will represent an outbound destination, and columns represent the price catalog (e.g., ADULT_ONEWAY, CHILD_ONEWAY, FULL_10TRIP, FULL_MONTHLY, BIKE_ONEWAY). Each cell will clearly display its active pricing status in the standard U.S. currency format, and routes that are not currently offered or active at that station display distinctive visual badges (e.g., NOT_SOLD or N/A) rather than ambiguous blank fields, preventing confusion between un-configured fares and free fares.

The operator can click directly into any price cell to enter edit mode. As values are adjusted, the interface visually highlights modified cells with a "pending draft" state, giving the operator an immediate visual log of all uncommitted modifications on the active page. The operator can discard pending edits on a cell-by-cell basis or reset the entire grid back to the current platform state before executing a save. Once satisfied with the adjustments, the operator clicks "Save Rate Changes". A confirmation dialog presents a concise summary of all pending rate changes (specifying Origin, Destination, Product, Old Price, and New Price) before final submission. Upon confirmation, the application commits the changes through the backend and returns a real-time status alert confirming how many price rules were updated on the platform.

Because the physical ticket vending machines rely on exact decimal pricing rules, this epic enforces multi-tiered client side (Browser) and server side (Spring Boot) data validation, handling Caracal platform architectural constraints. The module implements real-time client-side validation to reject blank entries, negative numbers, and currency values with more than two decimal places. On submission, the backend converts grid modifications into transactional payload rows and

executes bulk price adjustments against the platform API, updating the live origin-destination rule sets immediately.

Epic 3 : Ticket Availability and Product Lifecycle Management

Lead: *Athan L*

This epic delivers granular administrative control over where and when ticketing products can be purchased across the transit network.

Operators can toggle individual ticket classes on or off across specific terminal rate tables, cleanly handling permanent phase-outs such as the network-wide discontinuation of Senior fares. Furthermore, the engine models temporal availability conditions using the platform's when-condition-create mechanism, enforcing recurring calendar windows such as monthly pass sales (active exclusively from the twentieth of the preceding month through the tenth of the current month) and segregating weekday-only commuter products from weekend-specific fares.

Epic 4 : Pre-Purchase Service Alerts and TVM Preview

Lead: *Tejas D*

This module overcomes a core backend limitation of Caracal's platform, which currently provides no native schema for persisting customer warning banners or station service hours.

This epic implements a local PostgreSQL schema to store alert definitions, inactive route schedules, and customizable button actions, complete with dual-language support for English and Spanish terminal displays.

When a passenger attempts to purchase a ticket for a weekday-only route on a weekend, the system displays an informative pre-purchase alert allowing advance purchase while clarifying schedule restrictions.

To resolve customer confusion over ambiguous 'Confirm / Dismiss' prompts, the module provides configurable button phrasing alongside a live canvas that accurately simulates the physical ticket vending machine screen before changes are staged.

Epic 5 : Configuration Review, Validation, and Publishing

Lead: *Lakshya J*

This epic provides a deployment safeguard that prevents corrupted or unverified configurations from propagating to physical vending machines.

The module delivers a visual staging interface that compares pending rate and availability edits against active production values, highlighting modifications before commit. It automates validation against baseline transit datasets, ensuring the systematic elimination of all fifty known catalog discrepancies currently present in the sandbox.

Once verified, the operator executes a catalog release via the catalog-publish endpoint, recording an immutable entry retrieved through audit-log-get to maintain full administrative transparency.

Technical Architecture & REST API Integration

The application is built on Java 25 utilizing the Spring Boot framework (Spring MVC, Spring Data JPA, and Spring Security) paired with Thymeleaf server-side templates, Bootstrap styling, and vanilla JavaScript for dynamic client-side interactions.

Persistent application data, including operator credentials, alert message schemas, operating schedules, and staging drafts—is stored in a managed PostgreSQL database hosted on Render.

The system communicates with Caracal's Dockerized Go backend via HTTP REST endpoints using Spring WebClient, utilizing a local development proxy to circumvent platform CORS restrictions and dynamically targeting isolated sandbox tenants. (API Documentation is available privately via Caracal's `student-kit` provided to the developers)